

- Phase 2 Progress Report: Multimodal RAG Pipeline - Infrastructure and Media Processing
 - Overview
 - 1. Enhanced Media Processing Pipeline for Lecture Video Ingestion
 - 1.1 Motivation and Background
 - 1.2 Pipeline Architecture: Seven Sequential Stages
 - 1.3 Dual Execution Modes
 - 1.4 Technical Stack
 - 1.5 Current Status and Integration Readiness
 - 2. CI/CD Workflow: Automating the GitHub to AWS Deployment Pipeline
 - 2.1 Motivation
 - 2.2 Containerization: Packaging Each Service as a Docker Image
 - 2.3 GitHub Actions Workflow Architecture
 - 2.4 AWS Authentication Strategy
 - 2.5 Current Status
 - 3. System Architecture Diagrams
 - 3.1 The Distinction Between Architecture and Deployment Diagrams
 - 3.2 System Architecture Diagram
 - 3.3 Deployment Diagram
 - 3.4 Remaining Diagrams
 - 4. Terraform Infrastructure as Code
 - 4.1 Overview and Rationale for Infrastructure as Code
 - 4.2 Module Organization
 - 4.3 Provisioned Resources in Detail
 - 4.4 Verified Terraform Outputs Recorded After Deployment
 - 4.5 Components Not Yet Provisioned (Remaining ~40%)
 - 4.6 Terraform State Backend Migration
 - 5. Summary and Upcoming Work

Phase 2 Progress Report: Multimodal RAG Pipeline - Infrastructure and Media Processing

Reporting Period: January 2026 - March 2026

Subject: Capstone Thesis - BK-MInD: AI-Powered Lecture Learning System

Overview

This report documents the engineering work completed during Phase 2 of the capstone thesis project. The overarching goal of Phase 2 is to translate the theoretical research and prototyping conducted in Phase 1 into concrete, working, and deployable software components. Phase 2 is notably more engineering-intensive than Phase 1, requiring decisions not only about AI model selection and pipeline design, but also about system architecture, cloud infrastructure, deployment automation, and long-term scalability.

Four major workstreams were executed in parallel during this phase. The first workstream focused on implementing a complete and robust media processing pipeline capable of transforming raw lecture video and audio recordings into a semantically searchable knowledge base, which constitutes the core data ingestion mechanism for the RAG system. The second workstream addressed Continuous Integration and Continuous Deployment (CI/CD) automation, ensuring that any code change pushed to the version control system would be automatically validated, containerized, and deployed to the production environment on AWS without manual intervention. The third workstream involved the systematic analysis and production of architectural diagrams that will serve as formal documentation for the thesis, covering both the logical software architecture and the physical deployment topology. The fourth workstream involved provisioning the actual cloud infrastructure on Amazon Web Services using Terraform as Infrastructure as Code, translating the deployment design into a real, operational environment.

Each section below describes what was built, the reasoning behind key design decisions, the technical depth of the implementation, and the current status of each workstream.

1. Enhanced Media Processing Pipeline for Lecture Video Ingestion

1.1 Motivation and Background

One of the primary contributions of this thesis is the ability to process raw lecture videos and audio recordings and convert them into a semantically searchable knowledge base that students can query in natural language. In Phase 1, the team investigated individual tools and techniques for audio transcription, frame extraction, and primitive retrieval. However, those experiments were isolated: there was no unified system connecting all the steps end-to-end, nor any standardized output format that downstream retrieval components could rely on.

Phase 2 therefore required building a consolidated, production-quality processing pipeline with the following design requirements: the pipeline must be fully automated from raw video input through to a queryable index without requiring manual intervention between steps; all intermediate and final outputs must carry rich metadata so that retrieved results can be traced back to their exact position in the source video; the pipeline must degrade gracefully when optional dependencies (such as a paid API key) are unavailable; and the design must be modular enough to be integrated into a web server as a background processing job in the next phase.

1.2 Pipeline Architecture: Seven Sequential Stages

The pipeline processes a video or audio file through seven well-defined stages. Each stage produces structured outputs consumed by the next, forming a linear but internally rich processing chain.

Stage 1 - Audio Extraction and Noise Reduction

The entry point of the pipeline accepts video files in common lecture recording formats such as MP4, AVI, MOV, and MKV, as well as standalone audio formats including WAV, MP3, and M4A. For video inputs, the audio stream is extracted and saved separately as a WAV file. Before the audio is passed to any transcription model, a two-step noise reduction process is applied. The first step trims leading and trailing silence, which is common in recorded lectures that often begin with setup time. The second step applies a high-pass filter to remove low-frequency ambient noise, such as air conditioning hum or microphone rumble, which can degrade transcription accuracy. The choice to include this preprocessing step was motivated by real observations during Phase 1 testing, where unfiltered audio produced meaningfully worse transcription quality on certain recordings.

Stage 2 - Automatic Speech Recognition with Quality Filtering

The cleaned audio is fed into OpenAI's Whisper model for transcription. One of the important design choices at this stage was to request word-level timestamps rather than only segment-level timestamps. Word-level timestamps are necessary because the downstream chunk-to-frame alignment step (Stage 5) depends on precise temporal references to associate visual frames with the text being spoken at that moment. This would not be possible with coarse segment-only timestamps.

Beyond this, two quality filters are applied at the segment level to discard low-confidence speech: segments where the model estimates a high probability that no speech was present are filtered out, as are segments where the average log-probability of the predicted tokens falls below an acceptable threshold. These filters help ensure that filler sounds, background noise mistakenly transcribed as words, or heavily muffled speech do not pollute the downstream index. The output of this stage is a complete, structured transcript with timestamps at the word and segment levels, along with several derived formats including plain text subtitles (SRT) and WebVTT, the latter being directly usable by browser-based video players for caption display.

Stage 3 - Token-Aware Semantic Chunking

Retrieval-Augmented Generation systems do not index full transcripts as single documents. Instead, a transcript must be divided into chunks small enough for a language model to process within its context window, yet semantically coherent enough to constitute a meaningful answer unit. This stage implements a token-aware chunker that splits the transcript while respecting sentence boundaries. The maximum chunk size is set to 100 tokens, with a configurable overlap between adjacent chunks to preserve cross-sentence context. Importantly, each resulting chunk inherits precise metadata from the segments it was derived from: its token count, the start and end timestamps of the corresponding audio, and references back to the original transcript segments. This metadata is later used during retrieval to display the exact timestamp range in the video to the user when a matching chunk is returned.

Stage 4 - Frame Extraction with Perceptual Hash Deduplication

Lecture videos exhibit a particular characteristic that distinguishes them from general video content: long periods of near-static visual content. A lecturer may spend several minutes on a single slide, during which consecutive video frames are visually near-identical. If all frames were naively extracted and indexed, the index would be

dominated by redundant visual information, increasing storage costs and degrading retrieval signal quality.

To address this, frames are extracted at one frame per second by default using a video processing library, and then each frame is evaluated against the previous one using perceptual hashing (pHash). Perceptual hashing produces a compact fingerprint of the visual content of an image in a way that is robust to minor pixel-level variations. Two frames with a perceptual hash similarity above 0.95 are considered duplicates; the second frame is discarded. Only visually distinct frames are retained and saved, each with a metadata record containing its timestamp, frame number, perceptual hash value, and storage path. In practice, this deduplication step substantially reduces the number of stored frames for lecture recordings, often by 60% to 80% depending on the pacing of the lecture.

Stage 5 - LLM-Based Chunk Description Generation

This stage represents the most technically novel contribution of the media pipeline and reflects a deliberate architectural decision about how to improve retrieval quality. Raw lecture transcripts are often informal in their language: a lecturer might say "so this thing over here, this algorithm we talked about last week" rather than "the Naive Bayes classification algorithm." A student asking "what is Naive Bayes?" would receive a low similarity score against the raw transcript chunk, even though the content is directly relevant.

To bridge this semantic gap, each chunk is enriched with a natural-language description generated by a language model (GPT-4o-mini via the OpenAI API). This description is a concise, formal summary of what the chunk covers conceptually, written in the kind of language a student would use when searching. For example, a raw chunk might be transcribed speech about "this formula here", while the generated description would state "This segment explains the likelihood calculation in Naive Bayes and how prior probabilities are combined with evidence." When the enhanced description is included in the embedding, it dramatically improves the semantic relevance of retrieved chunks.

Alongside description generation, this stage performs frame-to-chunk temporal alignment: each retained frame is associated with the text chunk whose timestamp range contains that frame's timestamp. This builds a direct link between the visual and textual streams of the lecture, enabling a retrieval result to carry both the relevant text passage and the slide or board image from that moment in the lecture.

The entire stage is designed to fail gracefully. If the OpenAI API key is not present in the environment, this stage is skipped without interrupting the pipeline. The system continues with raw chunk text only, producing a functional but less semantically rich index. This design choice was made to ensure the pipeline remains fully testable and demonstrable during development without requiring API costs.

Stage 6 - Hybrid Retrieval Index Construction

With enhanced chunks available, the pipeline builds a searchable index using a hybrid retrieval strategy. Two complementary retrieval methods are combined:

The first is sparse retrieval using the BM25 algorithm. BM25 is a well-established term-frequency and inverse-document-frequency ranking function that is particularly effective for exact-match and keyword-focused queries. It handles technical terminology efficiently: if a student searches for "gradient descent," BM25 will rank chunks that literally contain that phrase very highly.

The second is dense retrieval using sentence embeddings. A pre-trained sentence transformer model is used to encode all chunks into fixed-dimensional dense vectors, which are then indexed using FAISS for efficient approximate nearest-neighbor search. Dense retrieval captures semantic similarity: if a chunk talks about "minimizing the loss function by adjusting weights," it may not match the exact phrase "gradient descent" but the dense embeddings will recognize it as semantically related.

A hybrid retriever combines the ranked lists from both methods using a tunable mixing coefficient (default: equal weighting). The final index along with all chunk metadata and document mappings is saved to disk, enabling fast reloading for subsequent queries without reprocessing the original video.

Stage 7 - Semantic Query Interface

The final output of the pipeline is a query-ready retrieval interface. When given a natural-language query, the system performs hybrid retrieval over the entire indexed corpus, returns the top-ranked chunks, and attaches to each result the matching text, its generated description, the source video filename, the precise timestamp range, and any associated visual frames. This level of provenance in the results is what allows the frontend to display not just a text answer but also the exact video moment in which the relevant explanation occurred, serving as both an answer and a direct reference into the original lecture.

1.3 Dual Execution Modes

The pipeline was implemented with two usage modes to serve different purposes. A batch processing mode enables the pipeline to be pointed at a directory of video files and processes them all sequentially with detailed console progress logging, intended for bulk ingestion of course materials. An interactive web interface, built with Streamlit, allows a user to upload a single video and observe the processing progress through each stage in real time, viewing intermediate outputs such as the transcript, extracted frames, and generated descriptions as they become available. The Streamlit interface was built primarily to support interactive debugging, rapid experimentation, and demonstration to stakeholders during development.

1.4 Technical Stack

Capability	Technology Used
Audio extraction from video	MoviePy
Audio preprocessing and noise reduction	librosa
Automatic Speech Recognition	OpenAI Whisper
Video frame extraction	OpenCV
Frame perceptual deduplication	imagehash (pHash algorithm)
LLM chunk enrichment	OpenAI GPT-4o-mini API
Sparse keyword retrieval	rank-bm25 (BM25Okapi)
Dense semantic retrieval	sentence-transformers + FAISS
Interactive demonstration interface	Streamlit

1.5 Current Status and Integration Readiness

The complete seven-stage pipeline has been implemented and verified end-to-end using real Coursera lecture videos as test inputs. All stages produce the expected output structures including clean audio, structured transcripts with timestamps, deduplicated frames, LLM-enriched chunks with frame associations, and a functional searchable RAG index. Semantic query tests against the index return contextually

relevant chunks with correct timestamp metadata. The pipeline is currently self-contained as a standalone module. The next step is to deploy it as an asynchronous background task within the main FastAPI backend, triggered when a user uploads a video file through the web interface.

2. CI/CD Workflow: Automating the GitHub to AWS Deployment Pipeline

2.1 Motivation

As the system matures past the prototype stage, the team needs a reliable, repeatable process for taking code changes from developers and getting them running in the production environment. Without automation, every code update requires a developer to manually rebuild Docker images, re-tag and push them to the container registry, update the cloud service configuration, and monitor the rollout. This manual process is slow, inconsistent, and introduces human error into a step that directly affects the stability of the live system. Furthermore, it creates a bottleneck: only team members with the correct cloud credentials and local tooling can deploy.

A Continuous Integration and Continuous Deployment pipeline solves this by making deployment a side effect of code review. When a developer merges code into the main branch, the entire build, test, publish, and deployment sequence runs automatically in the cloud without any further developer action.

2.2 Containerization: Packaging Each Service as a Docker Image

Before any deployment automation is meaningful, every service must be packaged in a portable, self-contained format. Docker containers were chosen for this, and a multi-stage build strategy was applied to both services to produce the smallest and most secure possible runtime images.

For the backend, a Python 3.11 Alpine Linux base was used. The build proceeds in two stages: a dependency installation stage that resolves and caches all Python packages from the requirements specification, and a final production stage that copies only the

application code and pre-built dependencies without any build tooling. The FastAPI server is exposed on port 5000. Health check endpoints are configured so that the load balancer can probe container readiness after startup. The resulting image is lean and does not carry any compilers, build tools, or unnecessary system utilities that would expand the attack surface.

For the frontend, a two-stage build was used: the first stage uses a full Node.js 20 environment to execute the Vite build process, compiling the React application into optimized static HTML, CSS, and JavaScript assets. The second stage is a minimal Nginx Alpine image, which serves those static assets at roughly 13 megabytes, compared to over 600 megabytes if the Node.js runtime were included. The Nginx configuration was written to handle single-page application routing (directing all unknown paths to the React entry point), proxy API requests to the backend container, apply gzip compression for faster asset delivery, and include security-oriented HTTP headers including HSTS, Content-Security-Policy, and X-Frame-Options.

2.3 GitHub Actions Workflow Architecture

Three distinct automated pipelines were designed, each covering a different dimension of the system. Each pipeline is triggered automatically by Git events and executes on GitHub's cloud runners.

Backend Deployment Pipeline

This pipeline activates whenever code changes are pushed or a pull request is opened targeting the main or develop branches, specifically when those changes affect the backend application source. The pipeline is organized into two sequential jobs.

The first job handles building and publishing the container image. It begins by authenticating to AWS using OpenID Connect federation (described separately in Section 2.4), which grants it temporary credentials scoped to exactly the permissions needed for image publishing. It then authenticates to Amazon Elastic Container Registry, builds the backend Docker image with layer caching enabled to minimize build time on repeated runs, and pushes the result to the registry with two tags: one that precisely identifies the triggering Git commit SHA (enabling rollback to any specific release) and one that updates the mutable "latest" pointer. After pushing, it creates or updates an ECS task definition that references the newly published image URI.

The second job handles the actual deployment to the live cluster. This job is conditioned to run only on direct pushes to the main branch (not on pull requests), ensuring that only reviewed and approved code reaches production. It renders the updated task definition and issues a force-new-deployment command to the ECS backend service, replacing running containers with new instances pulling the fresh image. The pipeline then monitors the deployment by polling the ECS service state until all desired tasks are confirmed healthy and running, which verifies that the deployment completed without degradation.

Frontend Deployment Pipeline

The frontend pipeline follows the same structural pattern as the backend. The build job includes additional frontend-specific steps: it sets up a Node.js environment with dependency caching to avoid re-downloading packages on every run, installs dependencies reproducibly, runs ESLint as a non-blocking validation step (the build proceeds even if there are warnings, ensuring lint issues do not block deployments while still producing a visible record), and executes the Vite production build to generate the optimized static asset bundle before packaging it into the Nginx-based Docker image. The deployment job follows the same ECS force-new-deployment and stability-monitoring pattern as the backend.

Infrastructure Management Pipeline

The third pipeline manages changes to the cloud infrastructure itself. It activates when changes are pushed to the Terraform configuration directory. The pipeline is organized into three sequential jobs representing a structured review-and-apply workflow.

The planning job runs on all triggers, including pull requests. It initializes the Terraform working directory against the configured state backend, validates the configuration syntax and provider schema compliance, and generates a complete execution plan that shows exactly which resources would be created, modified, or destroyed. This plan is saved as a versioned artifact with a retention window for audit purposes. When running against a pull request, the pipeline posts a human-readable summary of the planned changes as a comment directly on the pull request, giving the team visibility into infrastructure impact before any change is approved.

The apply job runs only on direct pushes to the main branch and only if the planning job succeeded. It downloads the exact plan artifact generated in the previous job and applies it, ensuring that the infrastructure change reviewed during the plan phase is precisely what gets executed. Using a saved plan artifact in this way prevents the plan-

apply race condition in which the state of AWS resources could drift between the time a plan was generated and the time apply is run.

A post-apply verification job reads all Terraform output values and prints a structured summary of every provisioned resource identifier, the application URLs, and a resource count confirmation, providing a clear record that the apply completed as expected.

2.4 AWS Authentication Strategy

A critical security decision in the CI/CD design was how GitHub Actions runners would authenticate to AWS. The naive approach of generating long-lived AWS access keys and storing them as GitHub Secrets carries significant risk: if those secrets were inadvertently exposed, they would grant persistent access to the AWS account until manually rotated.

Instead, the pipeline uses OpenID Connect (OIDC) federated identity. In this model, AWS IAM is configured to trust GitHub's OIDC provider as an identity authority. When a pipeline run starts, GitHub generates a short-lived, cryptographically signed identity token identifying the specific repository and branch. The pipeline presents this token to AWS, which validates it against the trust configuration and issues temporary credentials scoped to a specific IAM role with only the permissions needed for that pipeline. These credentials are valid for the duration of the pipeline run and then expire automatically. No long-lived secrets are stored anywhere.

2.5 Current Status

All three workflow definitions are complete, reviewed, and stored in the repository. They are currently maintained in a deactivated state to prevent premature triggering before the initial infrastructure is fully validated by a manual deployment. The activation plan is: first push the initial Docker images to ECR manually, force new deployments in ECS, confirm that all ALB target groups report healthy, and verify that the application responds correctly on both the frontend and backend endpoints. Once this baseline is confirmed, the workflows will be activated and all subsequent deployments will be fully automated through the CI/CD system.

3. System Architecture Diagrams

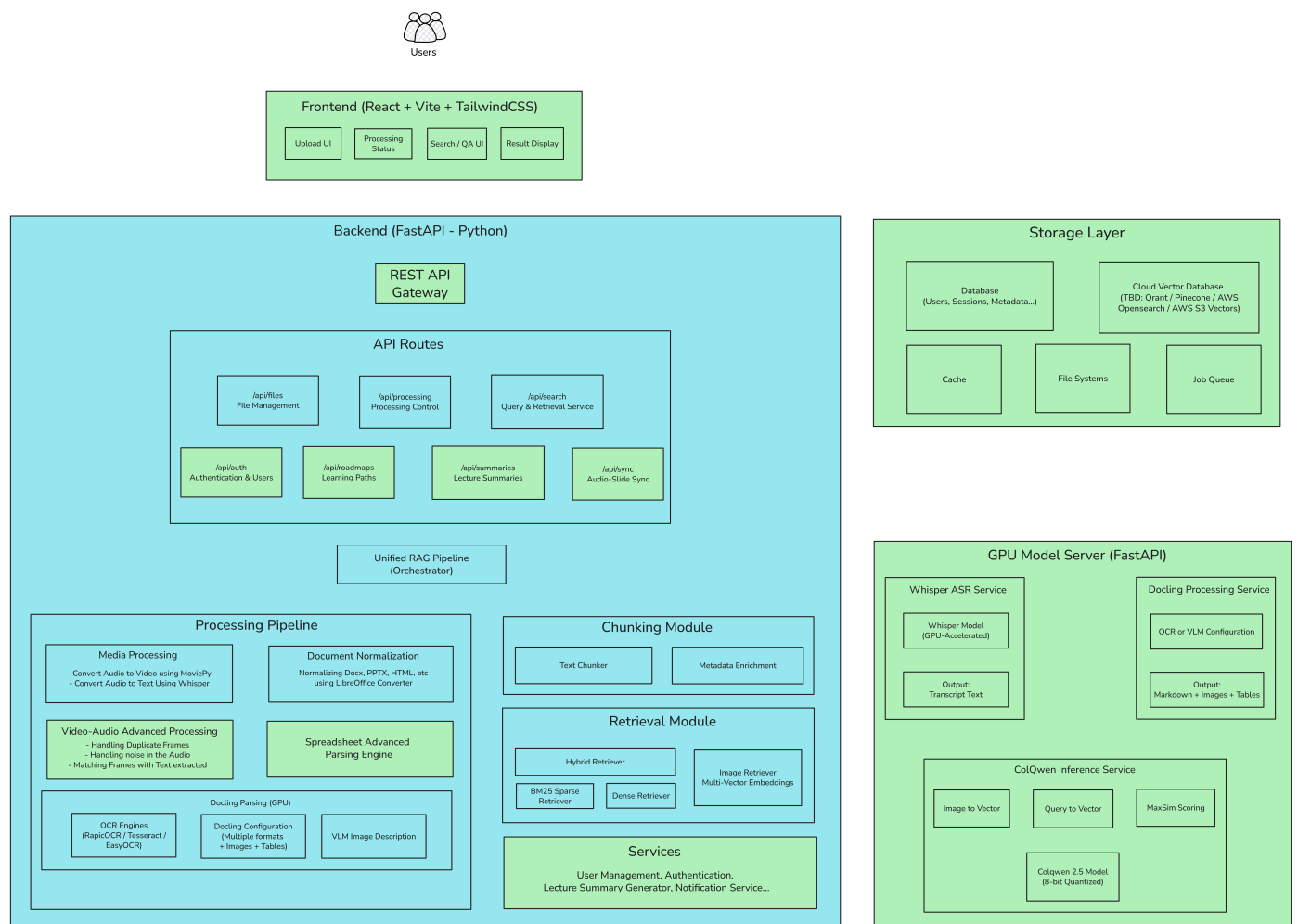
Two formal diagrams have been produced for the thesis to document the system design: a System Architecture Diagram and a Deployment Diagram. These two diagrams serve fundamentally different purposes and together provide a complete picture of both what the system is made of and where and how it runs.

3.1 The Distinction Between Architecture and Deployment Diagrams

A recurring source of confusion in software engineering documentation is treating the system architecture diagram and the deployment diagram as the same artifact. In this thesis, they are deliberately kept separate. The architecture diagram describes the logical structure of the software: what components exist, what responsibilities each component holds, and how they communicate with each other. It is abstracted away from physical infrastructure -- the same architecture could theoretically run on any cloud provider or on-premises. The deployment diagram, by contrast, describes the physical and cloud-level realization: which specific AWS services host which components, how network traffic is routed through security layers, which services are currently designed versus those not yet provisioned, and how the CI/CD system integrates with the infrastructure.

Both diagrams are required for the thesis because the engineering contribution of this project spans both dimensions. The software architecture communicates the AI system design; the deployment architecture demonstrates the engineering rigor of making the system production-ready on AWS.

3.2 System Architecture Diagram



The System Architecture Diagram was produced using Excalidraw and shows the full logical component structure of the BK-MInD system. The diagram is divided into four distinct visual areas: the Frontend, the Backend, the Storage Layer, and the GPU Model Server, each drawn as a separate bounded region to clearly communicate the separation of responsibilities.

The Frontend is labeled as a React, Vite, and TailwindCSS application. It contains four labeled sub-components arranged as a horizontal row: an Upload UI for submitting lecture materials, a Processing Status panel showing the current state of ingestion jobs, a Search / QA UI through which students pose natural-language questions, and a Result Display panel that presents retrieved answers. Conceptually, Users are positioned above the Frontend in the diagram, indicating the entry point of the entire system.

The Backend occupies the largest region and is labeled as a FastAPI Python application. At the top sits a central REST API Gateway box. Below it, an API Routes panel groups seven named route sets with their explicit URL path prefixes: the file management route at `/api/files`, the processing control route at `/api/processing`, the query and retrieval service at `/api/search`, authentication and user management at `/api/auth`, learning path generation at `/api/roadmaps`, lecture summary

generation at </api/summaries>, and audio-slide synchronization at </api/sync>. All seven route groups feed into a Unified RAG Pipeline Orchestrator, which acts as the central coordinator for all backend operations.

Beneath the orchestrator, the backend is organized into three parallel sub-regions. The first is the Processing Pipeline, which contains five processing sub-components. Media Processing handles video and audio inputs using MoviePy for audio extraction from video and Whisper for speech-to-text conversion. Document Normalization handles heterogeneous input formats such as DOCX, PPTX, and HTML by converting them into a uniform intermediate form using LibreOffice. Video-Audio Advanced Processing addresses the deduplication and matching work described in Section 1: handling duplicate frames, removing noise from the audio, and aligning extracted frames with the corresponding transcribed text. A Spreadsheet Advanced Parsing Engine handles structured tabular data inputs as a dedicated sub-component. The Docling Parsing (GPU) sub-component is the most complex, containing three nested components: OCR Engines supporting RapidOCR, Tesseract, and EasyOCR for scanned document handling; a Docling Configuration box that covers multiple document formats and handles embedded images and tables; and a VLM Image Description module that calls a vision-language model to generate natural-language descriptions of images found within documents.

The second sub-region is the Chunking Module, which appears as two adjacent components: a Text Chunker that splits normalized text into token-bounded segments, and a Metadata Enrichment component that annotates each chunk with timestamp, source reference, and other structured metadata before indexing.

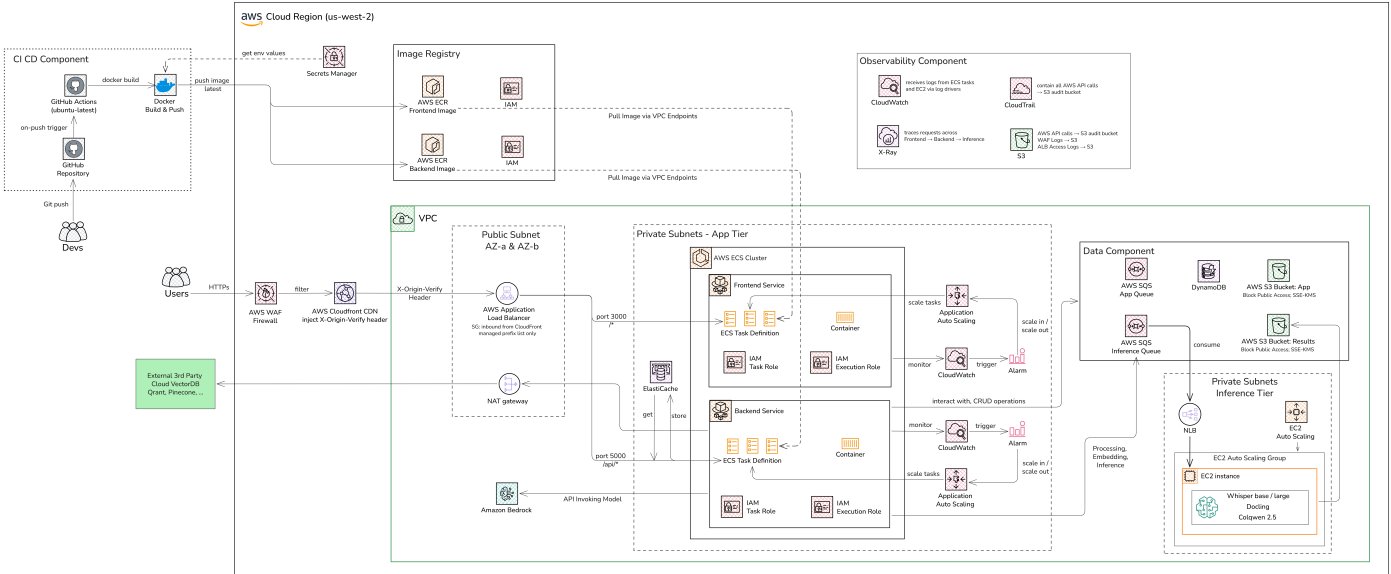
The third sub-region is the Retrieval Module, containing four retrieval strategies: a Hybrid Retriever as the top-level component that combines the outputs below; a BM25 Sparse Retriever for keyword-based exact matching; a Dense Retriever for semantic embedding-based matching; and an Image Retriever that operates on multi-vector embeddings, specifically the late-interaction ColQwen embeddings used for slide image retrieval. Below all three sub-regions sits a Services box listing: User Management, Authentication, Lecture Summary Generator, and Notification Services as the supporting service layer.

The Storage Layer is drawn as a separate bounded box to the top right of the diagram, communicating that it is shared infrastructure independent of the backend application itself. It contains five components: a Database for users, sessions, and metadata; a Cloud Vector Database with an explicit label noting the technology is still

to be decided, listing Qdrant, Pinecone, AWS OpenSearch, and AWS S3 Vectors as candidate options; a Cache component; a File Systems component for raw uploaded files; and a Job Queue for asynchronous task management.

The GPU Model Server is drawn as a fourth separate bounded box to the right of the diagram and is also labeled as a FastAPI application, running on dedicated GPU hardware. It exposes three named inference services. The Whisper ASR Service contains a Whisper Model (GPU-Accelerated) component and produces an output labeled Transcript Text. The Docling Processing Service contains an OCR or VLM Configuration component and produces an output labeled Markdown plus Images plus Tables, which covers the structured extraction of document content. The ColQwen Inference Service is the most detailed of the three, containing four labeled components: Image to Vector, Query to Vector, MaxSim Scoring, and the ColQwen 2.5 Model at 8-bit quantization. This service is what powers the visual embedding and retrieval capability, encoding both page images and natural-language queries into late-interaction embedding sequences and scoring their relevance using the MaxSim mechanism.

3.3 Deployment Diagram



The Deployment Diagram maps the logical architecture onto concrete AWS infrastructure. The diagram uses the standard AWS icon set and organizes all components into labeled bounded regions. The overall layout is divided into three zones: the CI/CD Component sitting outside the AWS boundary on the left, the AWS Cloud Region (us-west-2) occupying the center and right, and an External 3rd Party Cloud VectorDB block at the bottom left representing services hosted outside of AWS entirely.

The CI/CD Component shows the developer workflow that feeds images into AWS. Developers push code to the GitHub Repository, which triggers an on-push activation. GitHub Actions running on ubuntu-latest runners executes a Docker build and push job. The job retrieves environment configuration values from AWS Secrets Manager via a labeled arrow, builds the Docker image, and pushes it with the latest tag to the AWS ECR repositories in the Image Registry. This component makes explicit that the CI/CD process depends on Secrets Manager for any runtime configuration values needed at build time.

The Image Registry resides inside the AWS cloud boundary. It holds two Amazon ECR repositories: one for the Frontend image and one for the Backend image, each with an associated IAM configuration. Labeled arrows from the Image Registry to the ECS services inside the VPC are annotated "Pull Image via VPC Endpoints," indicating that container image pulls are routed through private VPC-internal endpoints rather than going out to the public internet.

The Observability Component appears in the top-right area of the diagram as a dedicated group. It contains four services: CloudWatch, which receives logs from ECS tasks and EC2 instances via log drivers; CloudTrail, which captures all AWS API calls and sends them to an S3 audit bucket; X-Ray, which traces requests across the Frontend, Backend, and Inference components for distributed request tracing; and S3, which consolidates audit logs including WAF logs and ALB access logs into dedicated buckets.

Inside the VPC, the infrastructure is split across two subnet tiers.

The Public Subnet, spanning availability zones AZ-a and AZ-b, contains four components handling the public-facing traffic edge. Users reach the system over HTTPS and first hit the AWS WAF Firewall, which applies web application firewall rules to filter malicious traffic. Filtered traffic is then passed to the AWS CloudFront CDN, which injects an X-Origin-Verify header into each request before forwarding it onward. The AWS Application Load Balancer sits behind CloudFront and is protected by a security group configured to accept inbound traffic only from the CloudFront managed prefix list, meaning direct requests bypassing CloudFront are not accepted. The ALB performs path-based routing: traffic on port 3000 matching the root and all non-API paths is forwarded to the Frontend Service, while traffic on port 5000 matching the `/api/` prefix is forwarded to the Backend Service. Also present in the public subnet is a NAT Gateway, which provides outbound internet access for the private subnet resources, and Amazon ElastiCache, which is positioned with labeled get and store

connections to both ECS services for fast in-memory caching of session state and frequently accessed data.

The Private Subnets App Tier contains the AWS ECS Cluster running the two application services. The Frontend Service group contains an ECS Task Definition with its running Container, an IAM Task Role and IAM Execution Role, a CloudWatch monitoring connection that feeds into an Alarm and triggers Application Auto Scaling to scale tasks in or out based on load. The Backend Service group follows the same structure: an ECS Task Definition plus Container, separate IAM Task Role and Execution Role, CloudWatch monitoring into an Alarm connected to Application Auto Scaling for independent scale-in and scale-out behavior. Additionally, Amazon Bedrock appears connected to the Backend Service via a labeled arrow reading "API Invoking Model," indicating that the backend can invoke foundation models through the AWS Bedrock managed service for certain inference tasks.

The Backend Service also has an outbound connection labeled "interact with, CRUD operations" pointing to the Data Component, covering all data read and write operations. A separate labeled connection "Processing, Embedding, Inference" points toward the Inference Tier, representing the jobs dispatched to the GPU cluster.

The Data Component is a bounded group on the right side of the diagram containing five services: an AWS SQS App Queue for general application-level async job management; DynamoDB for fast NoSQL key-value and document storage; an AWS S3 Bucket labeled App with Block Public Access and SSE-KMS encryption for raw uploaded application files; an AWS SQS Inference Queue that serves as the task bus between the backend and the inference tier, with a labeled "consume" connection to the EC2 instances; and an AWS S3 Bucket labeled Results, also with Block Public Access and SSE-KMS encryption, for storing processed inference outputs.

The Private Subnets Inference Tier is a separate bounded region below and to the right of the App Tier. It contains a Network Load Balancer (NLB) that distributes inference jobs, an EC2 Auto Scaling configuration, and an EC2 Auto Scaling Group containing an EC2 Instance. The EC2 Instance hosts three labeled inference services: Whisper base / large for ASR, Docling for document parsing, and Colqwen 2.5 for visual embedding and retrieval.

The External 3rd Party Cloud VectorDB block sits entirely outside the AWS boundary and is labeled with the candidates Qdrant and Pinecone, with a connection to the Backend Service. This represents the deliberate design decision to keep the vector

database as an externally managed service rather than self-hosting it within the AWS VPC, at least during this phase while the technology choice is still being evaluated.

3.4 Remaining Diagrams

In addition to the two completed diagrams, the thesis requires six more diagrams that have been identified and planned but not yet produced. These include a Use Case Diagram showing the actors and their interactions with the system, two or three Sequence Diagrams detailing critical interaction flows (specifically the document upload and processing flow, and the RAG query flow from user input through retrieval to LLM-generated response), an Entity-Relationship Diagram documenting the relational database schema, a Multimodal RAG Pipeline Architecture Diagram that illustrates the core AI contribution of the thesis in a flowchart style from raw input through indexing and retrieval to response generation, and a Component Diagram showing the internal module dependencies of the backend codebase. Among these, the Multimodal RAG Pipeline Architecture Diagram is considered the highest priority as it directly communicates the novel technical contribution of the project to thesis reviewers.

4. Terraform Infrastructure as Code

4.1 Overview and Rationale for Infrastructure as Code

The cloud infrastructure for Phase 2 is managed entirely using Terraform, a declarative Infrastructure as Code (IaC) tool. The decision to use IaC rather than provisioning resources manually through the AWS Management Console was driven by several engineering requirements. First, infrastructure defined in code is version-controlled alongside the application code, meaning every change to the cloud environment is traceable, reviewable, and reversible. Second, IaC enables the infrastructure configuration to be reproduced identically in a new environment (for example, a separate staging environment) with a single command. Third, it eliminates configuration drift, the common production problem where the actual state of cloud resources diverges from the documented state because manual changes were made without being recorded.

As of the successful deployment on February 15, 2026, Terraform has provisioned 36 AWS resources into the us-west-2 (Oregon) region. This represents approximately 60% of the complete infrastructure shown in the Deployment Diagram v2. The remaining 40% consists of the planned services that are architecturally identified but not yet provisioned.

4.2 Module Organization

The Terraform configuration is organized into a root module and five child modules, each encapsulating a logically cohesive slice of the infrastructure. This modular structure makes each component independently testable, reusable across environments, and easier to reason about when making targeted changes. The five modules are ECR (container image storage), IAM (access control), ALB (load balancing), ECS (compute orchestration), and autoscaling (dynamic capacity management).

```
terraform/
  main.tf          - Root configuration, module wiring, provider setup
  variables.tf     - All configurable input parameters
  outputs.tf       - Exported values for cross-module consumption and CI/CD
  terraform.tfvars - Actual environment-specific values (excluded from version
control)
  modules/
    ecr/           - ECR repository lifecycle and scanning policies
    iam/           - IAM execution roles and least-privilege policies
    alb/           - Application Load Balancer, listener rules, target groups
    ecs/           - ECS cluster, Fargate task definitions, services, security
groups
  autoscaling/     - CPU, memory, and request-count based scaling policies
```

The root configuration sources the default VPC and its associated subnets from AWS account data sources and wires all module outputs together. All resources inherit a consistent set of tags (Environment, Project, ManagedBy) applied via the provider `default_tags` block:

```
provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Environment = var.environment
      Project      = var.project_name
      ManagedBy    = "Terraform"
    }
  }
}
```

```
}  
}
```

4.3 Provisioned Resources in Detail

ECR Repositories

Two Amazon Elastic Container Registry repositories were created to store versioned Docker images for the backend and frontend services. Both repositories are configured with image tag mutability set to `MUTABLE` (allowing the `latest` tag to be updated on each deployment), with vulnerability scanning enabled on every image push (scan-on-push), and with a lifecycle policy that retains the 10 most recently pushed images and automatically purges older ones to control storage costs. The resulting image URLs follow the pattern `381492273521.dkr.ecr.us-west-2.amazonaws.com/rag-pipeline-{service}:latest`.

IAM Roles and Policies

Separate IAM role pairs (execution role and task role) were created for the backend and frontend services, following the principle of least privilege. The execution role grants ECS the permissions necessary to pull images from ECR and to write log events to CloudWatch Logs on behalf of the container -- these permissions are needed by the ECS control plane before the container starts running. The task role is the identity assumed by the running container itself, and it carries the permissions the application code needs at runtime, such as reading and writing to S3. Separating these two roles ensures that infrastructure management credentials are never exposed to the application code.

Application Load Balancer

A single Application Load Balancer named `rag-pipeline-alb` was provisioned across all available subnets in the default VPC to ensure high availability. A security group permits inbound HTTP traffic on port 80 from anywhere and all outbound traffic. Two target groups register the ECS tasks as backends: one for the backend service on port 5000 and one for the frontend service on port 3000. A listener on port 80 with a path-based forwarding rule routes requests beginning with `/api/` to the backend target group and all other requests to the frontend target group. The ALB performs periodic health checks against each target group and removes unhealthy tasks from rotation automatically.

The provisioned ALB DNS name is `rag-pipeline-alb-1719237910.us-west-2.elb.amazonaws.com`, making the application accessible at `http://rag-pipeline-alb-1719237910.us-west-2.elb.amazonaws.com`.

ECS Cluster and Fargate Services

An ECS cluster named `rag-pipeline-cluster` was created with Amazon ECS Container Insights enabled, which routes performance metrics and utilization data to CloudWatch for dashboard visualization. Two Fargate services run within this cluster under two separate task definitions:

The backend service (`rag-pipeline-cluster-backend-service`) runs containers from task definition revision `rag-pipeline-cluster-backend:1`. Each task is allocated 512 CPU units (0.5 vCPU) and 1024 megabytes of memory. The desired task count is 2, meaning two container instances run simultaneously behind the ALB for redundancy. The service is deployed in Fargate mode, so no EC2 instances need to be managed.

The frontend service (`rag-pipeline-cluster-frontend-service`) runs containers from task definition revision `rag-pipeline-cluster-frontend:1`. Each task is allocated 256 CPU units (0.25 vCPU) and 512 megabytes of memory with a desired count of 2. The ECS task security group is configured to accept inbound traffic only from the ALB security group on the container port, and permits all outbound traffic for image pulling, API calls, and CloudWatch log delivery.

CloudWatch Log Groups

Two CloudWatch Log Groups were created, one per service, to capture all container output. Container logs are delivered using the built-in `awslogs` Docker logging driver, which requires no application-level changes and works automatically for any container writing to standard output or standard error. Log retention is set to 7 days, sufficient for debugging recent deployments while controlling storage costs.

Auto-Scaling Policies

Application Auto Scaling targets and scaling policies were attached to both ECS services. The backend service scales between a minimum of 1 task and a maximum of 10 tasks. The frontend service scales between 1 and 5 tasks. Scaling is driven by three metrics per service: CPU utilization (target: 70%), memory utilization (target: 80%), and ALB request count per target. Scale-out triggers after 60 seconds of sustained above-

target load. Scale-in waits for 300 seconds of below-target load before reducing capacity, preventing premature scale-in during brief load drops.

4.4 Verified Terraform Outputs Recorded After Deployment

Output Key	Value
ALB ARN	arn:aws:elasticloadbalancing:us-west-2:381492273521:loadbalancer/app/rag-pipeline-alb/3a7175f2b74dc665
ALB DNS Name	rag-pipeline-alb-1719237910.us-west-2.elb.amazonaws.com
Application URL	http://rag-pipeline-alb-1719237910.us-west-2.elb.amazonaws.com
ECS Cluster ARN	arn:aws:ecs:us-west-2:381492273521:cluster/rag-pipeline-cluster
ECS Cluster Name	rag-pipeline-cluster
Backend ECR URL	381492273521.dkr.ecr.us-west-2.amazonaws.com/rag-pipeline-backend
Frontend ECR URL	381492273521.dkr.ecr.us-west-2.amazonaws.com/rag-pipeline-frontend
Backend Task Definition	arn:aws:ecs:us-west-2:381492273521:task-definition/rag-pipeline-cluster-backend:1
Frontend Task Definition	arn:aws:ecs:us-west-2:381492273521:task-definition/rag-pipeline-cluster-frontend:1
Backend Service URL	http://rag-pipeline-alb-1719237910.us-west-2.elb.amazonaws.com/api
Default VPC ID	vpc-06455a7fc73c8d965

4.5 Components Not Yet Provisioned (Remaining ~40%)

The following components are present in the Deployment Diagram v2 as planned and are not yet provisioned via Terraform. Each represents a distinct Terraform module to be added:

Amazon RDS (PostgreSQL): The relational database for persistent storage of user accounts, uploaded document metadata, processing job status, and session information. This requires a dedicated module that provisions a Multi-AZ database instance, associated subnet groups, and parameter groups. The database must be placed in private subnets, requiring a VPC redesign as a prerequisite.

Amazon ElastiCache (Redis): The in-memory cache for session token storage, API response caching for frequently requested content, and background job state management. A cache module will provision a Redis cluster in private subnets.

Amazon SQS: Message queues for decoupling the asynchronous media processing pipeline from the API endpoint that accepts uploads. When a user uploads a video, the API should immediately respond and enqueue a processing job, rather than blocking the request while the 7-stage pipeline runs. A dedicated module will provision the queue and a dead-letter queue for failed jobs.

Amazon Cognito: User pool and identity pool for authentication, replacing the basic authentication currently stubbed in the FastAPI backend. Cognito handles user registration, login, JWT issuance, and token refresh flows, eliminating the need to implement these security-sensitive features in application code.

AWS CloudFront and WAF: HTTPS termination, DDoS mitigation, and CDN distribution for global performance. CloudFront will sit in front of the ALB and serve frontend static assets from edge caches. WAF will enforce rate limiting and block common web attack patterns. These are planned for a combined CDN module.

EC2 g4dn.xlarge GPU Instance: The GPU inference server hosting Whisper ASR, Docling parsing, and ColQwen 2.5. This requires an EC2 module configuring the instance type, Deep Learning AMI, storage, security group, and a startup script deploying the three FastAPI inference services. This component is the most infrastructure-intensive remaining piece of the deployment.

Cloud Vector Database: The technology decision between Qdrant (self-hosted on ECS), AWS OpenSearch with k-NN plugin, and AWS S3 Vectors (the newest managed option) is currently under evaluation. Once finalized, a dedicated module will provision and configure the vector store and integrate it with the backend retrieval module.

4.6 Terraform State Backend Migration

The Terraform state is currently stored in a local file. Before the infrastructure CI/CD pipeline can be activated, the state must be migrated to a remote backend hosted on Amazon S3 with a DynamoDB table for state locking. The remote backend is already specified but commented out in the root Terraform configuration, ready to be enabled. This migration is a prerequisite for safe concurrent pipeline runs, because without state locking two simultaneous pipeline executions could corrupt the stored state.

5. Summary and Upcoming Work

The following table summarizes the completion status of all major Phase 2 workstreams covered in this report:

Workstream	Status	Notes
Enhanced Media Processing Pipeline	Complete	All 7 stages operational, tested with real lecture videos
Docker Containerization (Backend + Frontend)	Complete	Multi-stage builds producing lean production images
GitHub Actions CI/CD Workflows	Designed, pending activation	Deactivated pending baseline manual deployment verification
Terraform Infrastructure Provisioning	~60% complete	36 resources active; GPU, RDS, SQS, Cognito, CloudFront pending
System Architecture Diagram	Complete	Full Excalidraw diagram produced covering all 4 logical layers
Deployment Diagram v2	Complete	Active vs. planned infrastructure clearly distinguished with numbered flows
Remaining Thesis Diagrams	Planned, not yet produced	Use Case, Sequence, ERD, RAG Pipeline, Component, Activity diagrams

The immediate next steps for Phase 2 completion are as follows. The first priority is to push the initial Docker images to ECR, perform a manual force-new-deployment on both ECS services, verify that all ALB target groups report healthy, and confirm that the frontend and API respond correctly on the published ALB URL. Once this baseline deployment is confirmed, the GitHub Actions workflows can be activated and all subsequent code changes will flow automatically through the CI/CD pipeline. In parallel, the media processing pipeline will be integrated into the FastAPI backend as an asynchronous background task, triggered by file upload events. The Terraform state will be migrated to the remote S3 backend before the infrastructure pipeline is enabled. Finally, the six remaining thesis diagrams will be produced, beginning with the Multimodal RAG Pipeline Architecture Diagram as the highest-priority artifact for communicating the core AI contribution of the project.